MINOR PROJECT REPORT

ON

INSURECLAIM VISION

Submitted in partial fulfilment of the Requirements for the award of the degree of

Bachelor of Technology

In

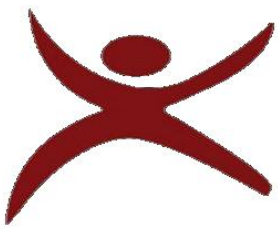
Computer Science and Engineering

By

Pullareddy Sandeep	**O211007**
Marri Kiran Kumar	**O211082**

Under the Guidance of

Ms. B SHIREESHA



Department of Computer Science and Engineering

RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES-AP

ONGOLE CAMPUS, 2025-2026.



CERTIFICATE

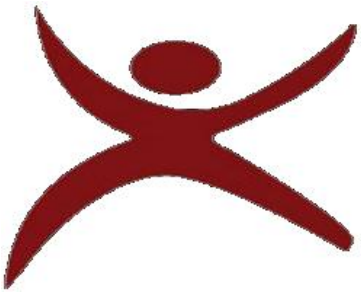
This is to certify that the project entitled "**InsureClaim Vision**" being submitted by **Pullareddy Sandeep** bearing the Id number **O211007** and **Marri Kiran Kumar** bearing the Id number **O211082** in partial fulfilment of the requirements for the End Semester Lab Examinations and for award of the degree of the Bachelor of Technology in Computer Science and Engineering to Rajiv Gandhi University of Knowledge Technologies is a record of bonafide work carried out by them under my guidance and supervision from DEC 2025 TO MAR 2026. The results presented in this project have been verified and found to be satisfactory. The results embodied in this project report have not been submitted to any other University for the award of any other degree.

Ms. B Shireesha,
Assistant Professor,
Department of CSE,
RGUKT ONGOLE.

Mrs. P Sindhu,
Head of the Department,
Department of CSE,
RGUKT ONGOLE.

RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



BONAFIDE CERTIFICATE

This is to certify that the project report entitled "InsureClaim Vision" submitted by Pullareddy Sandeep (O211007), Marri Kiran Kumar (O211082) in partial fulfilment of the requirement for the award of Bachelor of Technology in Computer Science and Engineering is a record of bonafide project work carried out under my supervision during the academic year 2025-2026.

We are indebted to Ms. B. Shireesha (Assistant Professor), our project guide, for conscientious guidance and encouragement to accomplish this project. We are extremely thankful and pay our gratitude to Mrs. P. Sindhu, Head of the Department CSE, for her valuable guidance and support on the completion of this project.

The report hasn't been submitted previously in part or full to this or any other university or institution for the award of any degree.

Ms. B Shireesha,
Assistant Professor,
Department of CSE,
RGUKT ONGOLE.

Mrs. P Sindhu,
Head of the Department,
Department of CSE,
RGUKT ONGOLE.

APPROVAL SHEET

This Report "**INSURECLAIM VISION**" Submitted by **P. SANDEEP** (O211007), **M. KIRAN KUMAR** (O211082), to **Ms B SHIREESHA** (Assistant Professor) is approved for the degree of Bachelor of Technology in Computer Science and Engineering.

Examiner(s):

_____

_____

Supervisor(s):

_____

_____

Date:

Place:

DECLARATION

We hereby declare that the project work entitled “INSURECLAIM VISION” submitted to the Rajiv Gandhi University Of Knowledge Technologies AP, ONGOLE Campus in partial fulfilment of the requirements for the award of the degree of MINOR PROJECT in Computer Science and Engineering is a record of an original work done by us under the guidance of Ms B SHIREESHA, Assistant Professor, Department of Computer Science and Engineering and this project work has not been submitted to any other university for the award of any other degree.

Pullareddy Sandeep (O211007) _____

Marri Kiran Kumar (O211082) _____

Date :

Place:

ACKNOWLEDGEMENT

We convey our profound thanks and gratitude to our Director RGUKT ONGOLE CAMPUS **Dr. A.V.S.S KUMARA SWAMI GUPTA** for providing us an excellent academic climate which made this endeavour possible. We extend our sincere thanks to our Academic Dean **Mr M RUPAS KUMAR**, for his encouragement and constant help. It is our privilege to express a profound sense of respect, gratitude and indebtedness to our guide **Ms B SHIREESHA** Assistant Professor, Department of Computer Science and Engineering, Rajiv Gandhi University of Knowledge Technologies-A.P, ONGOLE Campus, for her indefatigable inspiration, guidance, cogent discussion, constructive criticisms and encouragement throughout the dissertation work. We extend our sincere thanks to our H.O.D **Mrs P SINDHU** Assistant Professor, Department of CSE, RGUKT-ONGOLE.

Pullareddy Sandeep **O211007**

Marri Kiran Kumar **O211082**

ABSTRACT

InsureClaim Vision is a full-stack, AI-assisted motor insurance claim estimation platform designed to reduce the time taken for the first assessment of vehicle damage from days to seconds. A claimant or surveyor uploads photographs of a damaged vehicle, selects the vehicle class, preferred workshop type and pricing mode, and receives an itemised repair estimate with an instant pre-approval decision.

The frontend is developed using React 18, TypeScript, Vite and Tailwind CSS with Framer Motion animations. The backend is developed using Python FastAPI and exposes REST APIs for claim analysis, claim retrieval, claim listing and health checks. Each claim passes through a layered pipeline: an OpenCV perception layer that normalises the image, applies CLAHE contrast enhancement, extracts damage regions with Canny edges and maps the point of impact on a clock dial; a multimodal reasoning layer that sends the processed image to the Google Gemini Vision model to identify damaged parts, a severity score from 1 to 6 and a REPAIR or REPLACE decision; and a deterministic pricing engine that looks up part and labour costs from CSV rate cards and applies 18% GST.

A key feature of InsureClaim Vision is that money is never decided by the language model. The AI only describes the damage, while all costs come from a fixed catalogue, so the same damage always produces the same estimate. Claims below a configurable threshold of Rs. 50,000 are marked AUTO_APPROVE, while larger claims are routed to MANUAL_REVIEW. The system also generates a gradient heatmap and detection overlay for human auditors, stores every claim in SQLite and exports a professional PDF claim invoice.

Overall, InsureClaim Vision demonstrates how computer vision, large multimodal models and rule-based business logic can be combined in a single maintainable system to make motor claim estimation faster, more consistent and easier to audit.

Contents

1 INTRODUCTION 1
1.1 About Our Project 1
1.1.1 Key Functional Highlights 1
1.2 Motivation of the Project 2
1.3 Problem Definition 2
1.4 Objectives of the Project 3
2 LITERATURE SURVEY 4
2.1 Literature Review 4
3 ANALYSIS 6
3.1 Existing System 6
3.2 Proposed System 6
3.3 Hardware Requirement Specification 7
3.4 Software Requirement Specification 7
3.4.1 Overall Description 8
4 SYSTEM DESIGN 10
4.1 Architecture 10
4.1.1 Operational Flow Across Layers 10
4.2 UML Diagrams 11
4.2.1 Use Case Diagram 11
4.2.2 Object / Data Diagram 12
4.2.3 Class Diagram 13
4.2.4 Sequence Diagram 14
4.2.5 Activity Diagram 14
4.2.6 State Machine Diagram 16
4.2.7 Component Diagram 17
4.3 Statistical Analysis 18
5 IMPLEMENTATION 20
5.1 Modules 20
5.2 Module Description 20
5.3 Technologies Used 21
5.4 Sample Code 22
5.4.1 Image Upload and Validation 22
5.4.2 Contrast Enhancement with CLAHE 22
5.4.3 Gemini Vision Reasoning Call 22
5.4.4 Estimate Calculation and Approval 23
5.4.5 Part Cost Lookup with Fallbacks 23
5.4.6 Claim Persistence 23
5.5 Data Design Notes - Perception and AI Domain 23
5.6 Data Design Notes - Pricing Domain 24

5.7 Frontend Feature-Wise Implementation 24
5.7.1 Landing Page 24
5.7.2 Header and Backend Health Check 25
5.7.3 Estimator Page and Image Uploader 25
5.7.4 Pipeline Tracker 25
5.7.5 Heatmap Viewer 25
5.7.6 Detection Overlay 25
5.7.7 Estimate Panel 26
5.7.8 Approval Badge 26
5.7.9 PDF Invoice Export 26
5.8 Backend Feature-Wise Implementation 26
5.8.1 Server Bootstrap and Middleware 26
5.8.2 Claim Analysis API 26
5.8.3 Perception Pipeline 27
5.8.4 AI Reasoning Pipeline 27
5.8.5 Heatmap Refinement 27
5.8.6 Pricing Engine 27
5.8.7 Claim Persistence and Retrieval 28
5.8.8 Error Handling and Fallbacks 28
5.8.9 Security, Validation, and Reliability Notes 28
5.9 API Reference Summary 28
5.10 Additional Important Code 29
5.10.1 Axios Client and Route Contract 29
5.10.2 Vite Development Proxy 29
5.10.3 Clock-Dial Point of Impact 29
5.10.4 Detection Overlay on Canvas 29
5.10.5 PDF Invoice Generation 30
5.11 Reviewed Source Inventory 30
5.11.1 Frontend 30
5.11.2 Backend 30
6 SCREENSHOTS 31
7 CONCLUSION 39
8 FUTURE ENHANCEMENT 40
9 REFERENCES 41

1 INTRODUCTION

1.1 About Our Project

InsureClaim Vision is an online motor insurance claim estimation platform with a full-stack architecture. Its main purpose is to help claimants and insurance surveyors convert photographs of a damaged vehicle into an itemised repair estimate and an instant claim decision. The system supports two major personas: claimants (policyholders), who upload photos and receive an estimate, and insurance surveyors, who use the heatmap, overlay and stored claim records to review claims that need a human decision.

The frontend is implemented using React 18 with TypeScript and Vite, React Router for page navigation, Tailwind CSS for styling, Framer Motion for animation and Lucide icons. The backend is implemented using Python FastAPI served by Uvicorn, with Pydantic models acting as strict data contracts between layers. Image processing uses OpenCV and NumPy, the damage reasoning uses the Google Gemini Vision model through the google-generativeai SDK, pricing uses Pandas over CSV rate cards, and completed claims are stored in SQLite.

The project goes beyond a simple image classifier by combining three different kinds of intelligence. Classical computer vision measures image quality, finds high-gradient regions and estimates the point of impact. A multimodal language model identifies which parts are damaged and how badly. A deterministic financial engine then converts those decisions into rupees. In the reviewed codebase these layers are not isolated prototypes; they exchange validated Pydantic objects such as PerceptionResult, LLMResult and EstimateResult, which keeps the pipeline consistent and makes the design realistic for an insurance workflow.

1.1.1 Key Functional Highlights

- Drag-and-drop upload of one to six vehicle damage photographs with type and size validation.
- Vehicle class (hatchback, sedan, SUV, luxury), workshop type (independent or showroom) and pricing mode (OEM or aftermarket) selection that directly changes the estimate.

- OpenCV perception layer with normalisation, Gaussian denoising, CLAHE enhancement, Canny edges and clock-dial point-of-impact mapping.
- Gemini Vision reasoning that returns damaged parts, severity on a 1–6 scale, REPAIR / REPLACE decisions and the X/Y position of every damage point.
- Damage heatmap (JET colour map on Sobel gradient magnitude) refined around the AI-detected damage points.
- Detection overlay drawn on an HTML canvas with severity-coloured boxes and labels.
- Deterministic itemised estimate with part cost, labour cost, 18% GST and grand total.
- Automatic AUTO_APPROVE / MANUAL_REVIEW decision based on a configurable Rs. 50,000 threshold.
- Professional PDF claim invoice generated in the browser with jsPDF and AutoTable.

1.2 Motivation of the Project

The motivation for InsureClaim Vision comes from the slow and inconsistent nature of traditional motor claim inspection in India. After an accident, a policyholder usually waits for a surveyor to visit, inspect the vehicle, list damaged parts and prepare an estimate. This process can take several days, and two surveyors may produce different estimates for the same damage.

Our objective was to design a system where the first assessment of a claim is produced in seconds, while the final authority still stays with a human for expensive claims. We wanted the claimant to see exactly which parts were detected, why each part should be repaired or replaced, and how each rupee in the total was calculated. At the same time, we wanted surveyors to get visual evidence in the form of heatmaps and overlays so that they can quickly verify the AI output.

A second motivation was trust. Large language models are powerful at describing images, but they are not reliable calculators and can give different numbers for the same input. This project explores a practical approach where the model is used only for visual reasoning, and all money calculations are done by a fixed, auditable rule engine.

1.3 Problem Definition

Insurance companies receive a large number of small motor claims, and most of them involve common parts such as bumpers, headlamps, doors and fenders. Manually

inspecting every such claim is expensive and slow, which delays settlement and frustrates customers. In addition, manual estimates are hard to standardise because labour rates, part prices and repair-versus-replace judgements vary from person to person.

The problem addressed by InsureClaim Vision is therefore twofold. First, it must automatically understand vehicle damage from ordinary photographs: which parts are affected, how severe the damage is and whether each part should be repaired or replaced. Second, it must convert that understanding into a consistent, itemised and explainable cost estimate with a clear approval decision. The platform has to remain modular, explainable and accurate enough for academic demonstration as well as a realistic insurance pre-assessment workflow.

1.4 Objectives of the Project

- To develop a full-stack web platform for uploading vehicle damage images and receiving instant claim estimates.
- To build an OpenCV perception layer for image normalisation, enhancement, quality metrics and point-of-impact detection.
- To integrate a multimodal vision model that identifies damaged parts, severity and repair/replace decisions in a strict JSON format.
- To implement a deterministic pricing engine using part and labour rate cards with GST and workshop/pricing options.
- To provide visual evidence through gradient heatmaps and detection overlays for human auditors.
- To classify each claim automatically as AUTO_APPROVE or MANUAL_REVIEW using a configurable threshold.
- To persist every claim and allow it to be retrieved later by its claim ID.
- To keep the architecture clean enough for future extension into fraud detection, segmentation and mobile apps.

2 LITERATURE SURVEY

2.1 Literature Review

Automated vehicle damage assessment systems generally combine four main layers: an image capture interface, an image analysis layer, a damage-to-cost mapping layer and a decision layer. Commercial claim automation products, academic damage detection research and insurer-built inspection apps follow this broad pattern, even though their implementation choices differ.

[A] Image-Based Damage Detection: Early research on car damage assessment used hand-crafted features and later convolutional neural networks trained to classify damage types such as dents, scratches and glass breakage. These models need large labelled datasets for every part and damage type. InsureClaim Vision avoids training its own classifier and instead combines classical OpenCV features with a general multimodal model.

[B] Classical Image Pre-processing: Techniques such as Gaussian smoothing, histogram equalisation and edge detection are widely used to make images more consistent before analysis. Contrast Limited Adaptive Histogram Equalisation (CLAHE) improves local contrast without over-amplifying noise, and the Canny detector produces thin, reliable edges. The project uses both to highlight dents, creases and broken surfaces.

[C] Object Detection Models: Single-stage detectors such as the YOLO family can localise vehicles and parts in real time. They are fast but require fine-tuning on damage datasets to recognise damage itself. The project codebase includes an experimental YOLOv8 detection module, while the production pipeline relies on the vision-language model for part-level reasoning.

[D] Multimodal Large Language Models: Vision-language models such as Google Gemini accept an image together with a text instruction and can describe visual content in natural language or structured JSON. With a carefully designed system prompt and low temperature, they can act as zero-shot damage assessors. InsureClaim Vision uses this capability with a strict JSON output schema.

[E] Rule-Based Cost Estimation: Insurers and workshops estimate repairs using standard part price lists and labour tables. Rule-based estimation is transparent and repeatable,

which is important for audit and regulation. The project keeps this principle by computing every rupee from CSV rate cards instead of asking the model for prices.

[F] Explainable AI for Auditors: Saliency maps and heatmaps are commonly used to show where a model is looking. For claim review, a heatmap helps a surveyor quickly confirm whether the highlighted region matches real damage. The project generates a Sobel-gradient heatmap and refines it around the AI-reported damage points.

[G] API-First Backend Frameworks: Modern Python frameworks such as FastAPI provide asynchronous request handling, automatic validation with Pydantic and interactive OpenAPI documentation. This makes them a natural choice for machine-learning services where request and response shapes must be strictly controlled.

[H] UI Quality and Trust: Customer-facing insurance products invest heavily in clear interfaces because users must trust the result. The reviewed frontend shows deliberate emphasis on a step-by-step pipeline tracker, colour-coded severity, approval badges and downloadable invoices, demonstrating the importance of UX in AI-driven products.

3 ANALYSIS

3.1 Existing System

In most insurance settings, motor claims are still assessed through physical inspection. A claimant reports the accident, a surveyor is assigned, the vehicle is inspected at the workshop and a written estimate is prepared manually. Such approaches have several limitations:

- Long turnaround time between claim intimation and the first estimate.
- Inconsistent repair-versus-replace decisions and labour charges between surveyors.
- Little visual evidence recorded in a structured, searchable form.
- High operating cost for small claims that could be approved quickly.
- No instant feedback to the customer about the expected claim amount.

3.2 Proposed System

The proposed system, InsureClaim Vision, addresses these limitations with an integrated full-stack platform. It supports a complete lifecycle starting from image upload and vehicle details, to computer vision processing, AI damage reasoning, deterministic pricing, approval decision, claim storage and PDF export.

The proposed system introduces the following improvements:

- A layered pipeline (CV → AI reasoning → pricing) connected by validated Pydantic data contracts.
- Instant part-level damage identification with severity scores and repair/replace decisions.
- Deterministic, catalogue-based pricing that always produces the same amount for the same decisions.
- Configurable GST rate and approval threshold through environment variables.
- Visual heatmaps and detection overlays that make the AI output easy to verify.
- Automatic AUTO_APPROVE / MANUAL_REVIEW classification for faster settlement of small claims.
- Persistent claim records in SQLite and a downloadable PDF claim invoice.

The context diagram below shows the proposed system as a single process with its external entities: the user who submits damage photos, the Gemini Vision service that performs reasoning, and the SQLite storage that keeps claim records.

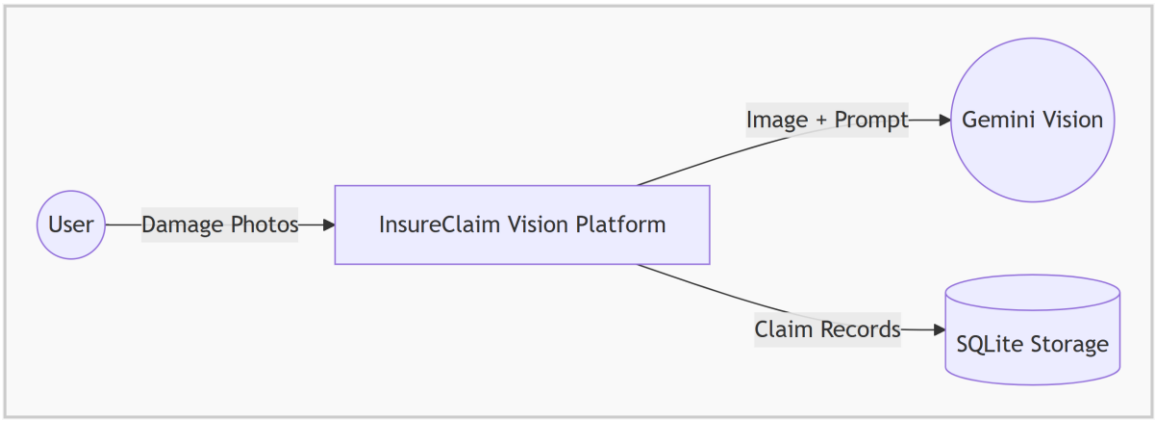


Figure 3.1: Context diagram of the proposed system

3.3 Hardware Requirement Specification

- Processor: Intel Core i3 / Ryzen 3 or above
- RAM: Minimum 4 GB, recommended 8 GB or more
- Storage: 2 GB free space for project files, Python packages, Node modules and stored heatmaps
- Internet: Required for package installation and for calls to the Gemini Vision API
- Display: 1366 × 768 or higher for comfortable use of the dashboard
- Camera / Phone: Recommended for capturing clear damage photographs

3.4 Software Requirement Specification

- Operating System: Windows, Linux, or macOS
- Frontend Runtime: Node.js 18+ with npm
- Frontend Stack: React 18, TypeScript, Vite 5, Tailwind CSS 3, Framer Motion, React Router, Axios, react-dropzone, Lucide icons
- Backend Runtime: Python 3.10+ with FastAPI and Uvicorn
- Computer Vision: OpenCV, NumPy, Pillow
- AI Model: Google Gemini Vision through the google-generativeai SDK (API key from Google AI Studio)
- Data Layer: Pandas for CSV rate cards, SQLite for claim storage

- Reporting: jsPDF and jsPDF-AutoTable
- Configuration: python-dotenv (.env file)

3.4.1 Overall Description

InsureClaim Vision operates as a client-server web application. The frontend communicates with the backend using REST APIs under the /api namespace, and the Vite development server proxies /api and /static requests to the FastAPI server on port 8000. The user uploads images as multipart form data together with vehicle class, workshop type and pricing mode.

For every claim the backend creates a unique claim ID, validates the files, and runs the primary image through the perception layer. The perception result, including the base64 normalised image and image quality metrics, is passed to the Gemini reasoning agent, which returns a list of part decisions. The heatmap is then refined around the reported damage points, the pricing engine computes line items, GST and the grand total, and the approval status is decided by comparing the total with the configured threshold.

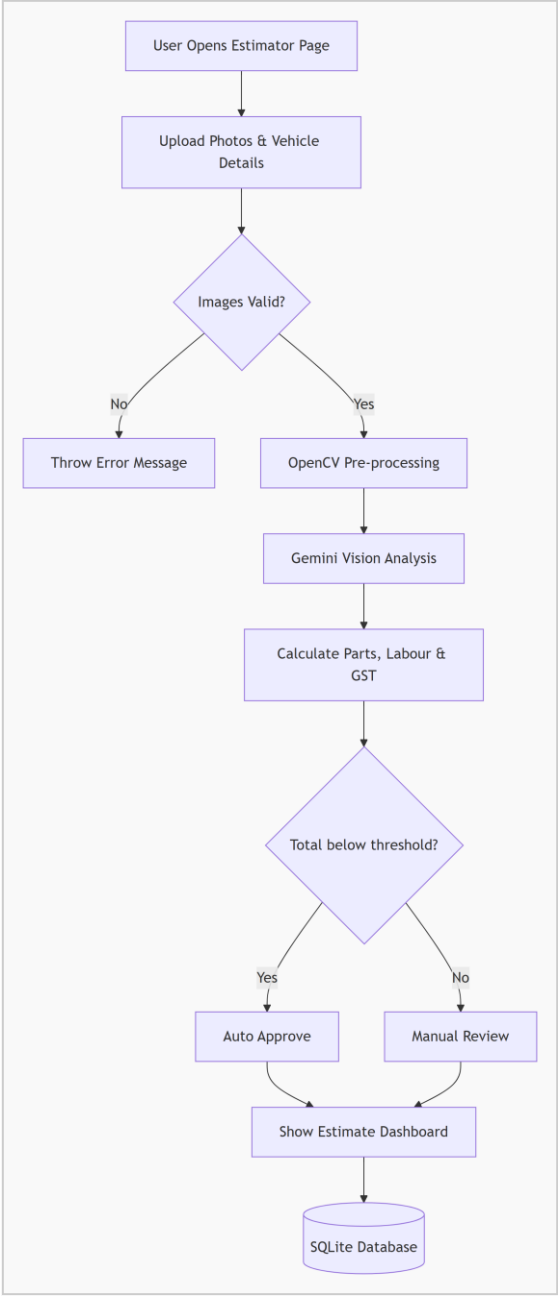


Figure 3.2: Workflow diagram of a claim request

Overall, InsureClaim Vision is designed as an explainable, configurable and extensible claim estimation platform. Its architecture keeps AI reasoning and financial calculation separate, which supports accurate evaluation, consistent pricing and easy auditing.

4 SYSTEM DESIGN

4.1 Architecture

The InsureClaim Vision architecture follows a layered web application model. The user interacts with React pages that manage route transitions and present UI states using component state. These pages call backend endpoints through a centralised Axios client with a two-minute timeout, because AI analysis can take several seconds.

The FastAPI backend organises business logic into independent modules. main.py handles routing, validation, CORS and static file serving; cv_processor.py implements the perception layer; llm_agent.py communicates with Gemini; pricing_service.py contains the financial engine; database.py stores claims; and models.py defines every data contract. Pricing data lives outside the code in two CSV files, so rates can be updated without changing Python code.

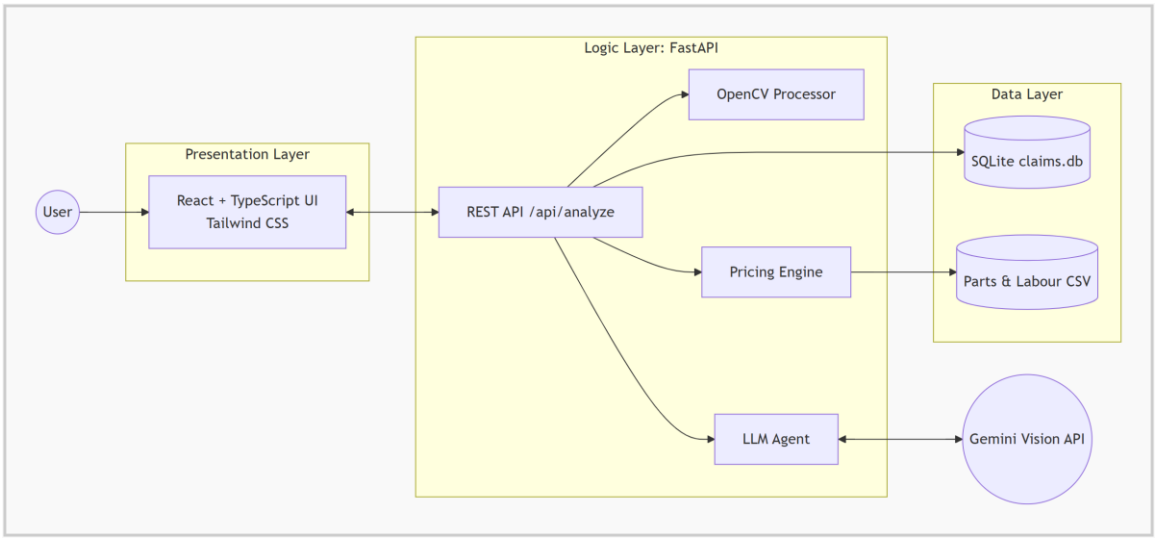


Figure 4.1: Layered architecture of the frontend, backend, AI service and data components

4.1.1 Operational Flow Across Layers

At runtime, a typical request travels through several coordinated layers. The EstimatorPage collects the images and options, the Axios client sends them to POST /api/analyze, and the orchestrator in main.py calls each layer in order. The CV processor returns a PerceptionResult, the LLM agent returns an LLMResult, the pricing service returns an EstimateResult, and these are combined into one ClaimAnalysisResponse that is saved to

SQLite and returned to the browser. Because every layer reads and writes typed objects, the flow remains understandable and each layer can be tested or replaced on its own.

4.2 UML Diagrams

Unified Modeling Language is used here as a documentation mechanism rather than a strict design-before-code artifact. The diagrams below were prepared from the actual modules and workflows in the codebase. Since the system is a pipeline, the diagrams highlight both user interaction and the flow of data between layers.

4.2.1 Use Case Diagram

The use case model shows that the claimant can upload photos, enter vehicle details, run the AI analysis, view the heatmap and overlay, view the itemised estimate and download the PDF invoice. The insurance surveyor uses the estimate and stored claim records to review claims that were flagged for manual review. Gemini Vision acts as an external system actor for the analysis use case.

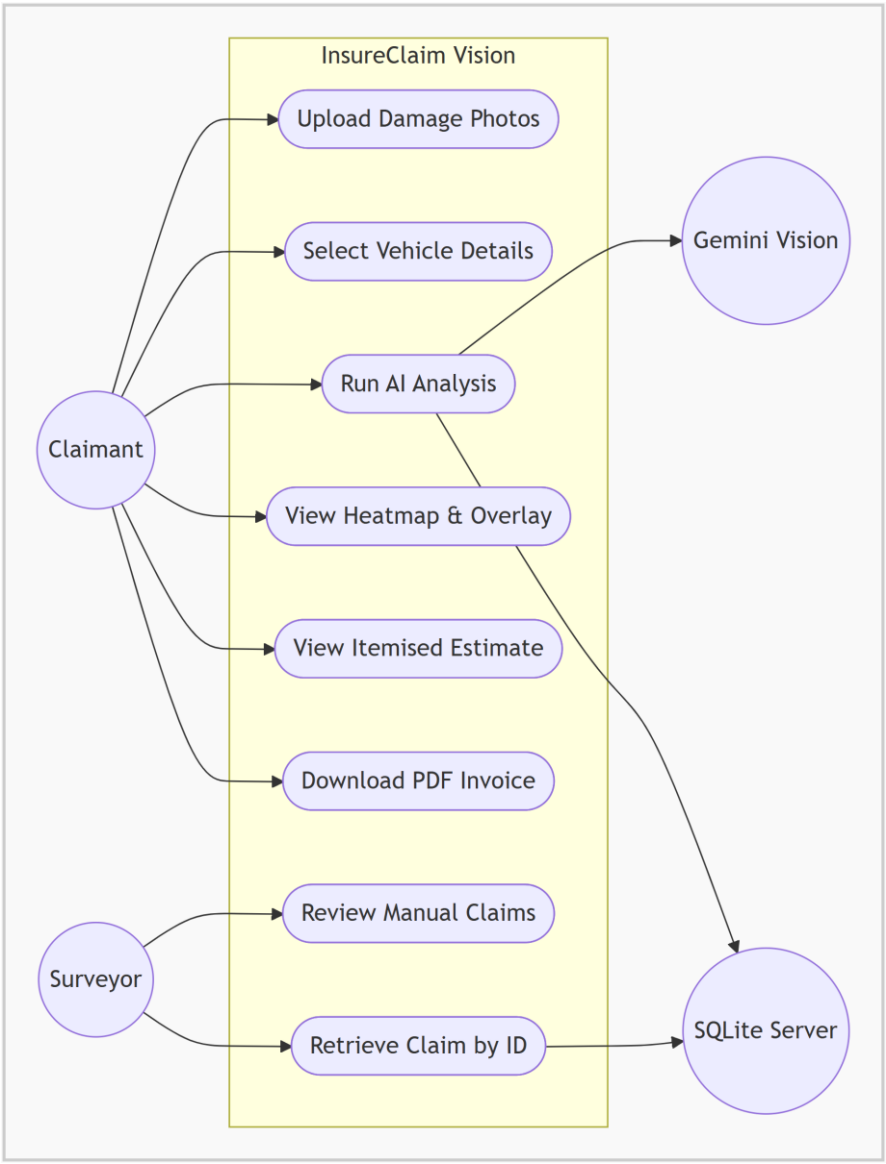


Figure 4.2: Use case relationships for claimants and surveyors

4.2.2 Object / Data Diagram

The data model is defined with Pydantic in models.py. The dominant objects are ClaimAnalysisResponse, PerceptionResult, ImageMetrics, DamageRegion, LLMResult, PartDecision, EstimateResult and EstimateLineItem. These entities capture both the visual evidence and the financial outcome of a claim. The entity relationship diagram below shows how a stored claim relates to its estimate line items and to the parts catalogue and labour rate tables used for pricing.

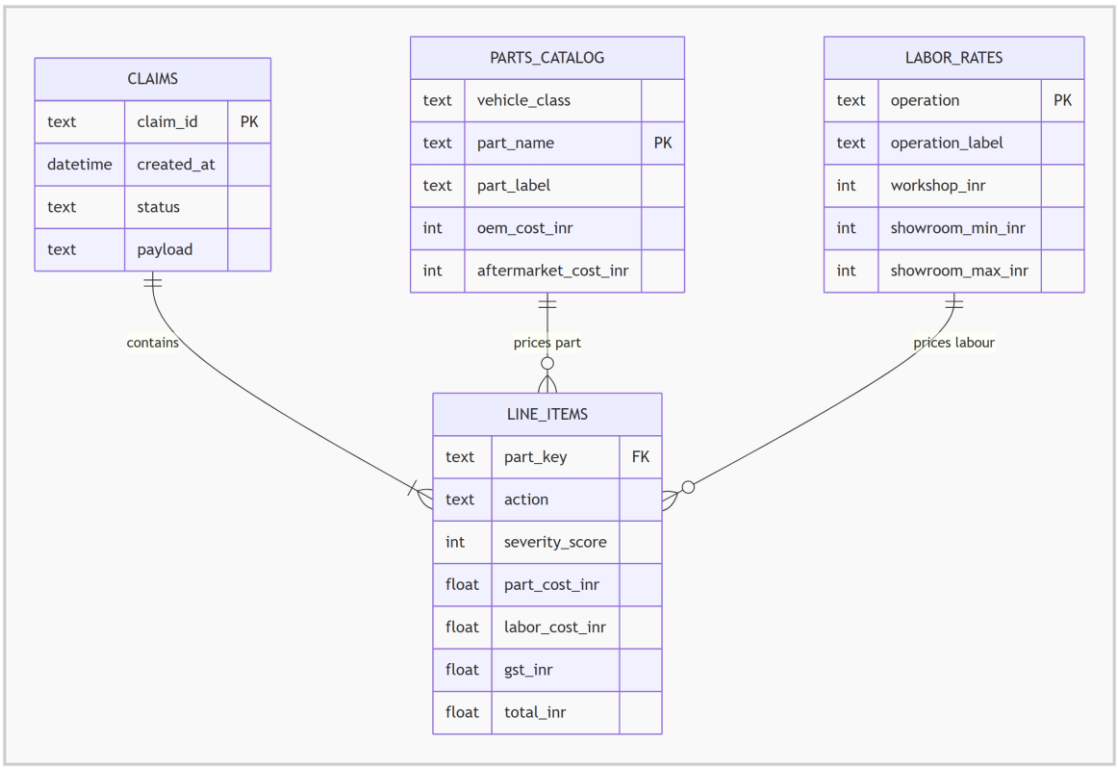


Figure 4.3: Entity relationship diagram of claim and pricing data

The key design benefit of this model is that each layer owns its own output object. The perception layer never knows about prices and the pricing layer never sees pixels. This reduces coupling and makes it possible to replace the AI model or the rate cards without changing other layers.

4.2.3 Class Diagram

Although the backend is written as Python modules with functions rather than classical object-oriented classes, its static structure can still be described in class-diagram terms. The conceptual classes are the FastAPI application, cv_processor, llm_agent, pricing_service and database, with models.py providing shared schemas and enumerations.

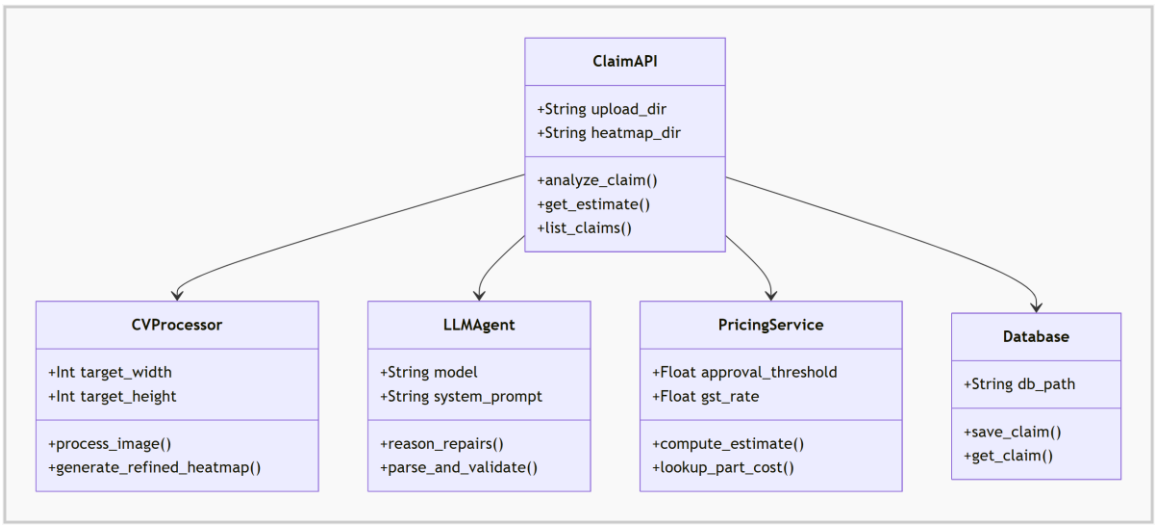


Figure 4.4: Conceptual class diagram of backend modules

4.2.4 Sequence Diagram

The most important runtime workflow in the system is a user submitting damage photographs and receiving an estimate.

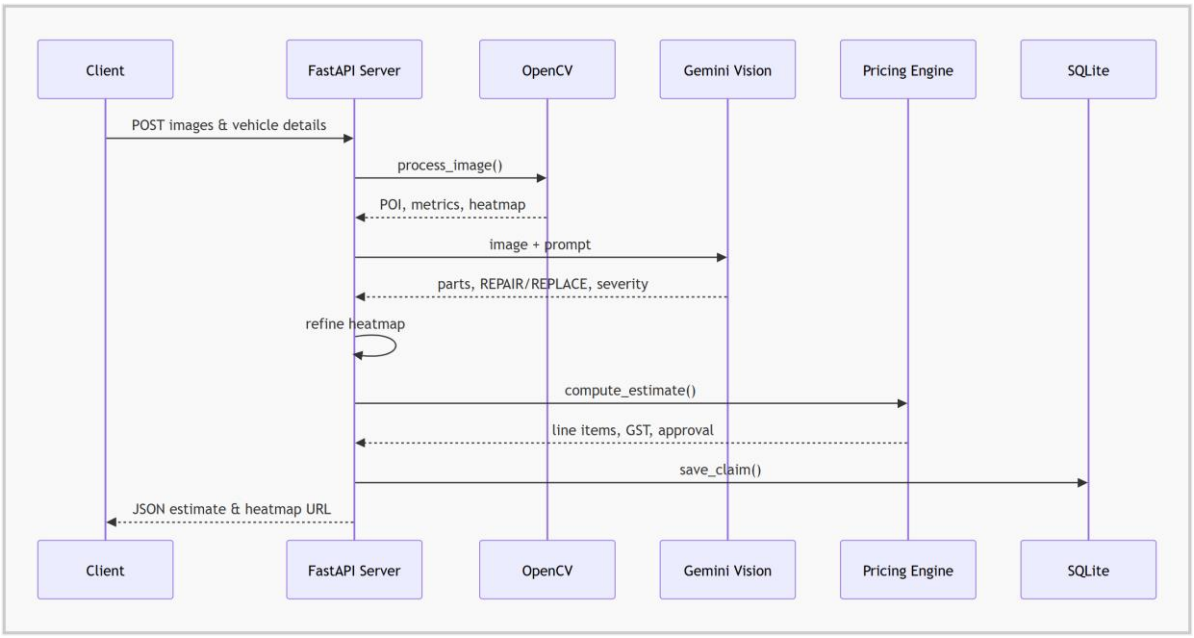


Figure 4.5: Sequence of events during claim analysis

4.2.5 Activity Diagram

The main activity flow for a claim can be represented as:

1. User opens the landing page and starts a claim.

2. User uploads one to six damage photos.

3. User selects vehicle class, workshop type and pricing mode.

4. Backend validates file type, file size and option values.

5. OpenCV normalises the image, measures quality and builds the first heatmap.

6. Gemini Vision identifies damaged parts, severity and repair/replace decisions.

7. The heatmap is refined around the AI-detected damage points.

8. The pricing engine calculates part, labour and GST for each part.

9. The total is compared with the threshold to decide AUTO_APPROVE or MANUAL_REVIEW.

10. The claim is saved, shown to the user and optionally downloaded as a PDF invoice.

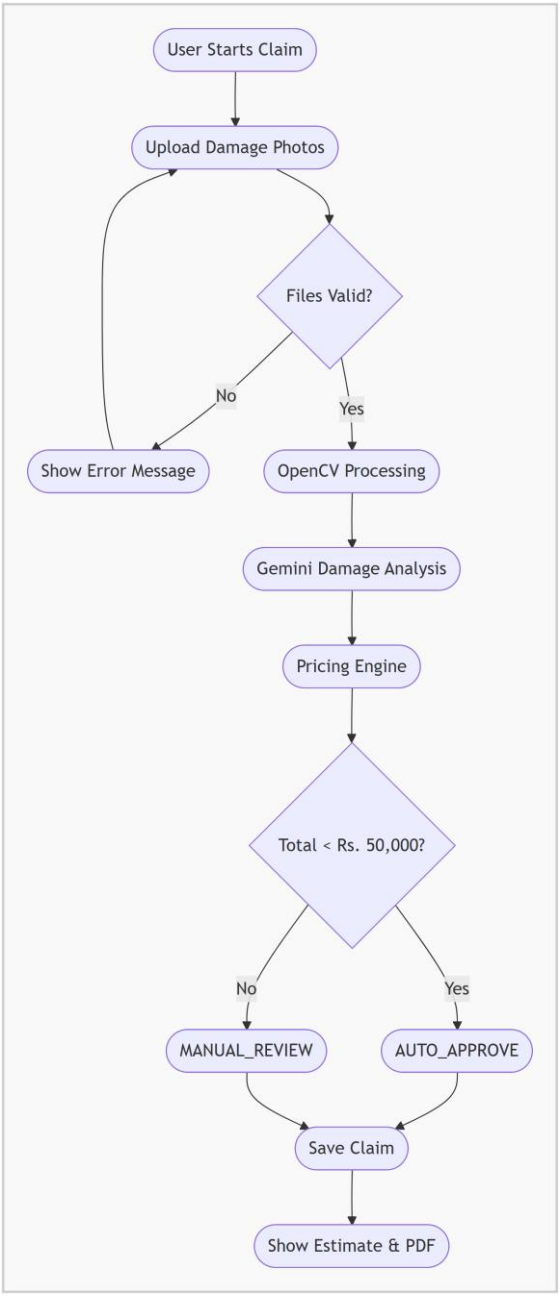


Figure 4.6: Activity diagram for a claim estimation session

4.2.6 State Machine Diagram

A claim moves through a compact state model. It starts as Submitted, becomes Validated when the files pass checks, Perceived after OpenCV processing, AI Assessed after Gemini reasoning and Priced after the pricing engine runs. From Priced it moves to Auto-Approved when the total is below the threshold or to Manual Review otherwise, and in both cases ends in the Stored state. Invalid uploads move directly to Rejected.

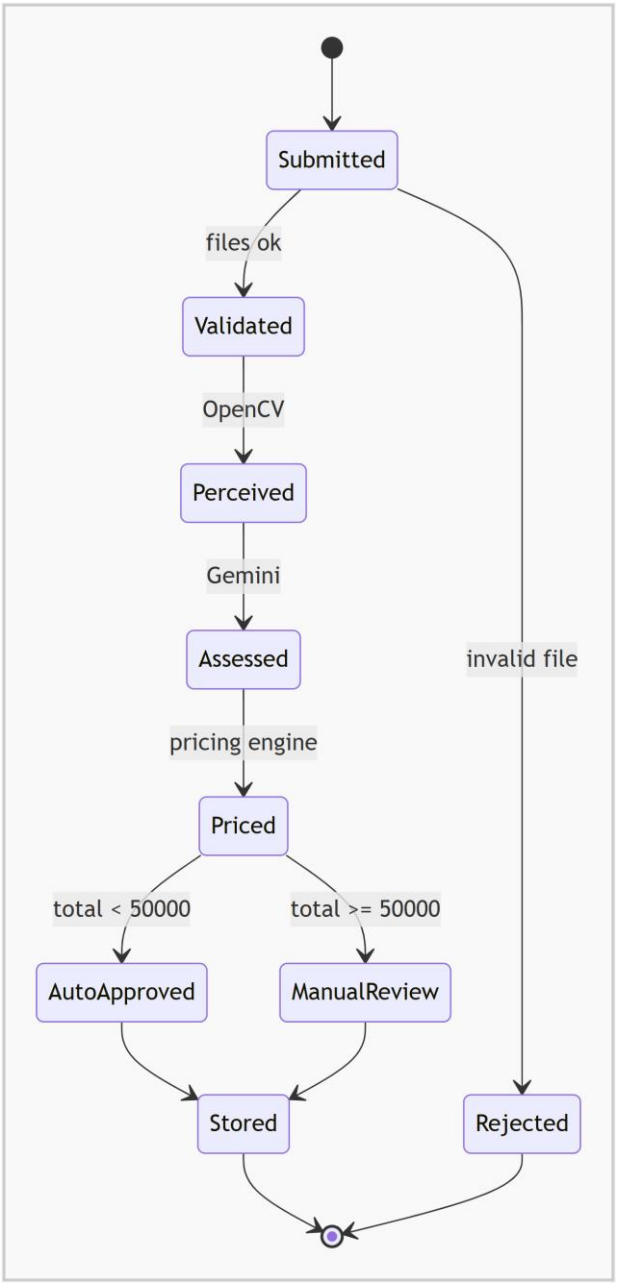


Figure 4.7: State machine of a claim

4.2.7 Component Diagram

The primary components are the React single-page application, the Axios API client, the FastAPI REST layer, the processing modules, the SQLite store and the external Gemini Vision service. Specialised frontend components include the image uploader, pipeline tracker, heatmap viewer, detection overlay, estimate panel and approval badge.

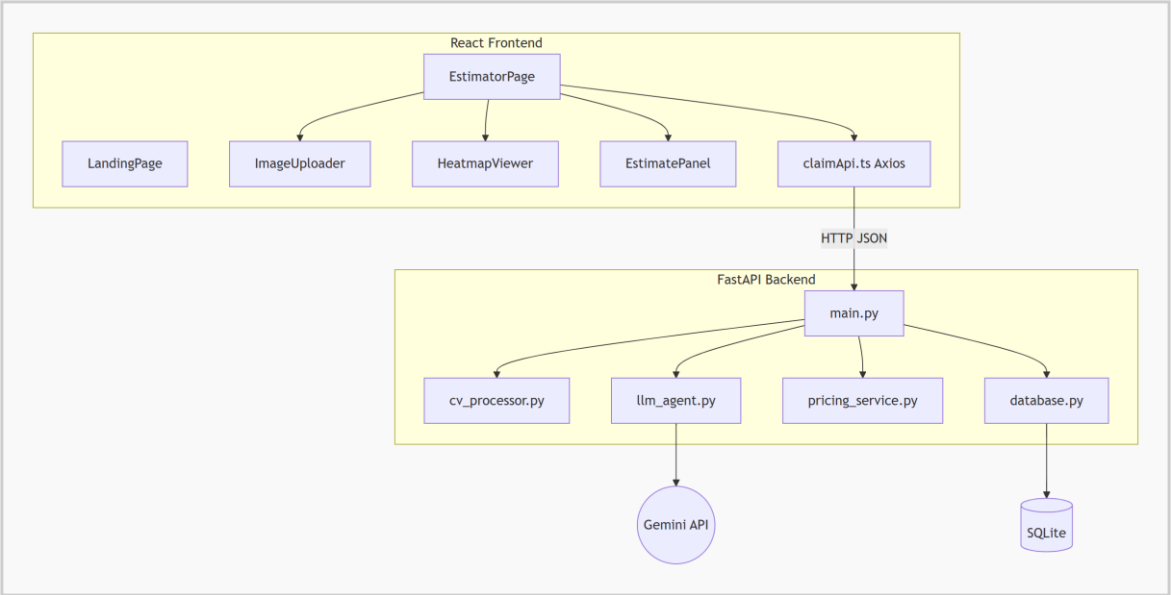


Figure 4.8: Component-level interaction between frontend, backend and AI service

4.3 Statistical Analysis

Unlike a pure machine-learning project, InsureClaim Vision's statistical analysis is best represented through codebase and feature metrics rather than only accuracy figures, because the AI model is used as a pre-trained service. The current implementation contains:

Metric	**Observed Value**
Frontend pages	2
Frontend components	7
Backend Python modules	7
Pydantic data models	9
Enumerations (damage type, decision, vehicle class, ...)	7
REST API endpoints	4
Parts catalogue entries (4 vehicle classes)	72
Labour operations in rate card	11
Total Python / TypeScript / CSS source lines	3,987 lines

These metrics show that the project is substantial enough to be treated as a complete mini-platform rather than a single-feature application.

Beyond raw counts, the source code also reflects meaningful feature depth. The backend contains a multi-step image processing pipeline, a carefully constrained AI prompt with output validation, a pricing engine with three levels of fallback, and thread-safe claim

persistence. The frontend contains animated pipeline status, canvas-based overlays, a side-by-side heatmap comparison and client-side PDF generation. This indicates that the project is not only broad in terms of file count, but also reasonably deep in terms of workflow implementation.

5 IMPLEMENTATION

5.1 Modules

1. Image Upload and Validation Module
2. Computer Vision Perception Module
3. AI Damage Reasoning Module
4. Heatmap and Detection Overlay Module
5. Pricing and Estimation Module
6. Approval Decision Module
7. Claim Storage and Retrieval Module
8. Report (PDF Invoice) Module

5.2 Module Description

Image Upload and Validation Module: Accepts one to six JPG, PNG or WEBP images with vehicle details, rejects non-image or corrupt files and validates option values against enumerations.

Computer Vision Perception Module: Normalises the image to 1024 × 768, removes noise, enhances contrast with CLAHE, measures brightness, blur and contrast, extracts damage regions and maps the point of impact.

AI Damage Reasoning Module: Sends the processed image and CV context to Gemini Vision and converts the JSON reply into validated part decisions with severity and coordinates.

Heatmap and Detection Overlay Module: Builds a JET-colour gradient heatmap, refines it around AI damage points and draws severity-coloured boxes on the original photo.

Pricing and Estimation Module: Looks up part cost by vehicle class and pricing mode, labour cost by operation and workshop type, and adds GST to produce line items and totals.

Approval Decision Module: Marks claims below the threshold as AUTO_APPROVE and the rest as MANUAL_REVIEW.

Claim Storage and Retrieval Module: Saves each response as JSON in SQLite and serves it back by claim ID or as a recent-claims list.

Report Module: Generates a formatted A4 PDF invoice with claim details, line items, totals and approval status in the browser.

5.3 Technologies Used

Technology	Purpose in InsureClaim Vision
React 18 + TypeScript	Builds the client-side interface with typed, composable pages and components.
Vite	Provides fast development, API proxying and bundling for the frontend.
Tailwind CSS	Used for responsive styling, glass-style cards, badges and gradients.
Framer Motion	Animations for the landing page, pipeline tracker and result panels.
react-dropzone	Drag-and-drop multi-image upload with type and count limits.
Axios	HTTP client with upload progress and timeout handling.
FastAPI + Uvicorn	Implements the asynchronous REST API and serves heatmap images.
Pydantic	Defines and validates the data contracts between all layers.
OpenCV + NumPy	Image normalisation, CLAHE, Canny edges, contours, Sobel heatmaps.
Google Gemini Vision	Multimodal reasoning for part identification, severity and repair decisions.
Pandas	Loads and queries the parts catalogue and labour rate CSV files.
SQLite	Lightweight storage for every analysed claim.
jsPDF + AutoTable	Generates the downloadable PDF claim invoice.
python-dotenv	Loads API key, GST rate, threshold and paths from the .env file.

5.4 Sample Code

5.4.1 Image Upload and Validation

```
@app.post("/api/analyze", response_model=ClaimAnalysisResponse)
async def analyze_claim(
    images: List[UploadFile] = File(...),
    vehicle_class: str = Form("hatchback"),
    workshop_type: str = Form("independent"),
    pricing_mode: str = Form("aftermarket"),
):
    claim_id = f"CLM-{time.strftime('%Y%m%d')}-{str(uuid.uuid4())[:6].upper()}-HYD"
    try:
        v_class = VehicleClass(vehicle_class.lower())
        w_type = WorkshopType(workshop_type.lower())
        p_mode = PricingMode(pricing_mode.lower())
    except ValueError as e:
        raise HTTPException(status_code=422, detail=str(e))
    if len(images) > 6:
        raise HTTPException(status_code=422, detail="Maximum 6 images per claim.")
```

5.4.2 Contrast Enhancement with CLAHE

```
def _apply_clahe(img_bgr: np.ndarray) -> np.ndarray:
    """Apply CLAHE to luminance channel (LAB color space)."""
    lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)
    l_channel, a_channel, b_channel = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=CLAHE_CLIP_LIMIT,
tileGridSize=CLAHE_TILE_GRID)
    l_enhanced = clahe.apply(l_channel)
    lab_enhanced = cv2.merge([l_enhanced, a_channel, b_channel])
    return cv2.cvtColor(lab_enhanced, cv2.COLOR_LAB2BGR)
```

5.4.3 Gemini Vision Reasoning Call

```
model = genai.GenerativeModel(model_name=GEMINI_MODEL,
system_instruction=SYSTEM_PROMPT)
image_part = {"mime_type": "image/jpeg", "data": perception.normalized_image_b64}

response = model.generate_content(
    contents=[image_part, user_message],
    generation_config=genai.types.GenerationConfig(
        temperature=0.1,                # near-deterministic structured output
        top_p=0.9,
        max_output_tokens=8192,
        response_mime_type="application/json",
    ),
)
decisions = _parse_and_validate(response.text.strip())
```

5.4.4 Estimate Calculation and Approval

```
for decision in llm_result.decisions:
    line_item = _price_decision(decision, parts_df, labor_df,
                                vehicle_class, workshop_type, pricing_mode)
    line_items.append(line_item)
    subtotal += line_item.subtotal_inr

gst_amount = round(subtotal * GST_RATE, 2)
grand_total = round(subtotal + gst_amount, 2)
approval = (ApprovalStatus.AUTO_APPROVE
            if grand_total < APPROVAL_THRESHOLD
            else ApprovalStatus.MANUAL_REVIEW)
```

5.4.5 Part Cost Lookup with Fallbacks

```
cost_col = "oem_cost_inr" if pricing_mode == "oem" else "aftermarket_cost_inr"

# Exact match: vehicle_class + part_name
row = df[(df["vehicle_class"] == vehicle_class) & (df["part_name"] == part_key)]
if not row.empty:
    return float(row.iloc[0][cost_col]), False

# Fallback 1: same part from any vehicle class
row_part = df[df["part_name"] == part_key]
if not row_part.empty:
    return float(row_part.iloc[0][cost_col]), True

# Fallback 2: average cost for this vehicle class
class_df = df[df["vehicle_class"] == vehicle_class]
return round(float(class_df[cost_col].mean()), 2), True
```

5.4.6 Claim Persistence

```
def save_claim(claim_id: str, status: str, payload: dict) -> None:
    """Persist a claim result."""
    with _lock:
        conn = _get_connection()
        conn.execute(
            "INSERT OR REPLACE INTO claims (claim_id, status, payload) VALUES (?,
?, ?)",
            (claim_id, status, json.dumps(payload)),
        )
        conn.commit()
        conn.close()
```

5.5 Data Design Notes - Perception and AI Domain

The perception and AI domains form the foundation of the platform. The PerceptionResult model stores the point of impact (for example Front-Left), the clock-dial orientation code, up to ten damage regions with area, centroid, bounding box and gradient intensity, the image quality metrics and the path of the generated heatmap.

The PartDecision model is intentionally strict. Instead of storing free text from the AI, it stores the part name, a snake_case part key that matches the pricing catalogue, a REPAIR or REPLACE enumeration, a severity score limited to 1–6, a justification of at most 200 characters and the X/Y position of the damage as percentages of the image size.

One important design decision is that the LLM output is always re-validated in Python. Severity is clamped to the 1–6 range, unknown decisions default to REPAIR and missing coordinates default to the image centre, so a malformed reply can never break the pricing layer.

5.6 Data Design Notes - Pricing Domain

The pricing subsystem is modelled as data rather than code. parts_catalog.csv stores the OEM and aftermarket cost of 72 part entries across hatchback, sedan, SUV and luxury classes, covering bumpers, hood, headlamps, tail lamps, fog lamps, doors, fenders, quarter panels, grille, roof panel and windshields. labor_rates.csv stores 11 labour operations with an independent-workshop rate and a showroom minimum and maximum rate.

A mapping table (PART_TO_LABOR_OP) links each part to its replacement operation, for example front_bumper to replace_bumper and headlamp_left to replace_lamp. Repairs always use the panel-repair operation, and showroom labour is calculated as the midpoint of the showroom range. Every line item records whether its cost was exact or estimated through a fallback.

5.7 Frontend Feature-Wise Implementation

5.7.1 Landing Page

The public entry layer is a marketing-style landing page with a full-screen video hero (“Drive Safe. Stay Protected.”), a coverage section describing collision damage, theft and fire, and third-party liability, and an animated features section explaining computer vision perception, multimodal reasoning and the dynamic financial engine. The “Start Your Claim Now” button navigates to the estimator.

5.7.2 Header and Backend Health Check

A shared header shows the InsureClaim Vision brand. On load, the application calls GET /api/health to confirm that the backend is reachable and that a Gemini API key is configured, so the user knows immediately if the AI service is unavailable.

5.7.3 Estimator Page and Image Uploader

The estimator page is the main working screen. The ImageUploader component uses react-dropzone to accept up to six JPG, PNG or WEBP photos, shows thumbnails with remove buttons, and collects vehicle class, vehicle make, preferred workshop (independent or showroom) and pricing mode (OEM or aftermarket).

This page is also where the project most clearly demonstrates product-level thinking. Instead of a single upload button and a number, the result area combines the approval badge, visual evidence and the itemised estimate in one view, so the user can understand both the decision and the reasons behind it.

5.7.4 Pipeline Tracker

The PipelineTracker component shows the four processing stages – Image Upload, CV Processing, Gemini Reasoning and Pricing Engine – with live status and upload progress, so the user can see what the system is doing during the few seconds of analysis.

5.7.5 Heatmap Viewer

The HeatmapViewer displays the original photo and the AI-refined heatmap side by side, together with the detected point of impact and image quality metrics such as brightness, blur score and contrast.

5.7.6 Detection Overlay

The DetectionOverlay component draws the uploaded image on an HTML canvas and places a severity-coloured box and label at every damage coordinate returned by the AI. Colours move from green for minor damage to red for severe damage.

5.7.7 Estimate Panel

The EstimatePanel lists every part with its action (REPAIR or REPLACE), severity pill, part cost, labour cost, GST and total, followed by the subtotal, GST and grand total. Parts priced through a fallback are marked as estimated so that reviewers can check them.

5.7.8 Approval Badge

The ApprovalBadge shows the claim ID, grand total, threshold, processing time and a clear green AUTO_APPROVE or amber MANUAL_REVIEW status at the top of the results.

5.7.9 PDF Invoice Export

The “Save PDF” action dynamically imports jsPDF and AutoTable and builds an A4 claim invoice with a dark header, claim reference, vehicle and workshop details, an itemised table, totals and the approval status. Because the PDF is generated in the browser, no extra server load is created.

5.8 Backend Feature-Wise Implementation

5.8.1 Server Bootstrap and Middleware

The backend server starts in main.py. Environment variables are loaded from .env before any module reads them, CORS middleware is configured so the frontend can call the API, the uploads directory is mounted at /static for heatmap images, and the SQLite database is initialised in the startup event. Running python main.py starts Uvicorn on port 8000 with auto-reload.

5.8.2 Claim Analysis API

POST /api/analyze is the orchestrator. It generates a claim ID, validates every option and file, runs the perception, reasoning, heatmap refinement and pricing layers in order, measures the total processing time, assembles the ClaimAnalysisResponse and stores it.

5.8.3 Perception Pipeline

process_image() decodes the bytes, letterboxes the image to 1024 × 768, applies a 5 × 5 Gaussian blur and CLAHE, computes image metrics (Laplacian-variance blur score, brightness, contrast), runs Canny edge detection (50/150), keeps contours larger than 200 px² as damage regions, computes an area-weighted centroid for the clock-dial point of impact and writes the initial heatmap. A blur score below 20 adds a warning to the response.

5.8.4 AI Reasoning Pipeline

reason_repairs() configures the Gemini SDK with the API key and sends the normalised image with a system prompt that defines visual extraction, part identification, a three-band severity engine (minor 1–2, moderate 3–4, severe 5–6), coordinate mapping and repair-versus-replace rules. JSON response mode and a temperature of 0.1 keep the output structured and stable.

After the call, the agent does more than return raw text. It strips any markdown, extracts the JSON array, validates every field, clamps values and builds PartDecision objects. This careful handling is important because it keeps the downstream pricing deterministic even when the model reply is imperfect.

5.8.5 Heatmap Refinement

generate_refined_heatmap() builds a mask around each AI-reported damage point and multiplies the Sobel gradient magnitude by this mask, so the final heatmap highlights only regions that the model also considers damaged.

5.8.6 Pricing Engine

compute_estimate() loads the two CSV files once (cached with lru_cache), prices each decision, adds GST at the configured rate and decides the approval status. The pricing is 100% deterministic and never asks the AI for a number.

5.8.7 Claim Persistence and Retrieval

database.py stores each claim as one row containing the claim ID, timestamp, status and the full JSON payload. GET /api/estimate/{claim_id} returns a stored claim and GET /api/claims returns the most recent claims. A threading lock protects concurrent writes.

5.8.8 Error Handling and Fallbacks

Each layer is wrapped in its own error handler: invalid input returns 400 or 422, AI failures return 503 and pricing problems return 500 with a clear message. If the Gemini free-tier rate limit is reached, the agent logs a warning and returns a clearly labelled mock result so that the interface can still be demonstrated. Heatmap refinement and database saving are non-fatal and only log warnings when they fail.

5.8.9 Security, Validation, and Reliability Notes

Several reliability-oriented decisions are visible in the backend. The API key is read from the environment and never sent to the browser, file types and sizes are checked before processing, option values are validated against enumerations, LLM output is re-validated before use and money is calculated only from trusted rate cards. Together, these choices improve trust in the platform and make the implementation stronger for academic review.

5.9 API Reference Summary

Endpoint	Method	Purpose
/api/health	GET	Check backend status and whether the Gemini key is configured.
/api/analyze	POST	Upload 1–6 images with vehicle details and receive the full AI estimate.
/api/estimate/{claim_id}	GET	Fetch a previously stored claim by its ID.
/api/claims	GET	List recent claims with ID, date and approval status.
/static/heatmaps/{file}	GET	Serve the generated heatmap image for a claim.

5.10 Additional Important Code

5.10.1 Axios Client and Route Contract

```
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || '/api',
  timeout: 120_000, // 2 minutes - CV + Gemini can be slow
})

const response = await api.post<ClaimAnalysisResponse>('/analyze', fd, {
  headers: { 'Content-Type': 'multipart/form-data' },
  onUploadProgress: (e) => {
    if (e.total && onProgress) onProgress(Math.round((e.loaded / e.total) * 100))
  },
})
```

5.10.2 Vite Development Proxy

```
export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/api':    { target: 'http://localhost:8000', changeOrigin: true },
      '/static': { target: 'http://localhost:8000', changeOrigin: true },
    },
  },
})
```

5.10.3 Clock-Dial Point of Impact

```
total_area = sum(r.area_px for r in regions)
wx = sum(r.centroid[0] * r.area_px for r in regions) / total_area
wy = sum(r.centroid[1] * r.area_px for r in regions) / total_area
cx_norm, cy_norm = wx / img_w, wy / img_h

h_sector = "Left" if cx_norm < 0.4 else "Right" if cx_norm > 0.6 else "Center"
v_sector = "Front" if cy_norm < 0.4 else "Rear" if cy_norm > 0.6 else "Mid"
```

5.10.4 Detection Overlay on Canvas

```
decisions.forEach((dec) => {
  const cx = (dec.x_percentage / 100) * canvas.width
  const cy = (dec.y_percentage / 100) * canvas.height
  const color = SEVERITY_COLORS[dec.severity_score] ?? '#94a3b8'
  const boxSize = Math.max(canvas.width, canvas.height) * 0.12
  ctx.strokeStyle = color
  ctx.lineWidth = 3
  ctx.strokeRect(cx - boxSize / 2, cy - boxSize / 2, boxSize, boxSize)
})
```

5.10.5 PDF Invoice Generation

```
const handleSavePDF = async () => {
  const { jsPDF } = await import('jspdf')
  const { autoTable } = await import('jspdf-autotable')
  const doc = new jsPDF({ orientation: 'portrait', unit: 'mm', format: 'a4' })

  doc.setFillColor(17, 24, 39)
  doc.rect(0, 0, doc.internal.pageSize.getWidth(), 38, 'F')
  doc.setFontSize(18)
  doc.text('MOTOR CLAIM ESTIMATE', 14, 14)
  // ... itemised table with autoTable(doc, {...}) ...
  doc.save(`motor-claim-${claimId}.pdf`)
}
```

5.11 Reviewed Source Inventory

5.11.1 Frontend

Frontend Area	**Main Reviewed Files**
Routing / Shell	App.tsx, main.tsx, index.css
Public Page	LandingPage.tsx, LandingFeatures.tsx
Estimation Flow	EstimatorPage.tsx, ImageUploader.tsx, PipelineTracker.tsx
Visual Evidence	HeatmapViewer.tsx, DetectionOverlay.tsx
Results / Reports	EstimatePanel.tsx, ApprovalBadge.tsx
API and Types	api/claimApi.ts, types/claim.ts

5.11.2 Backend

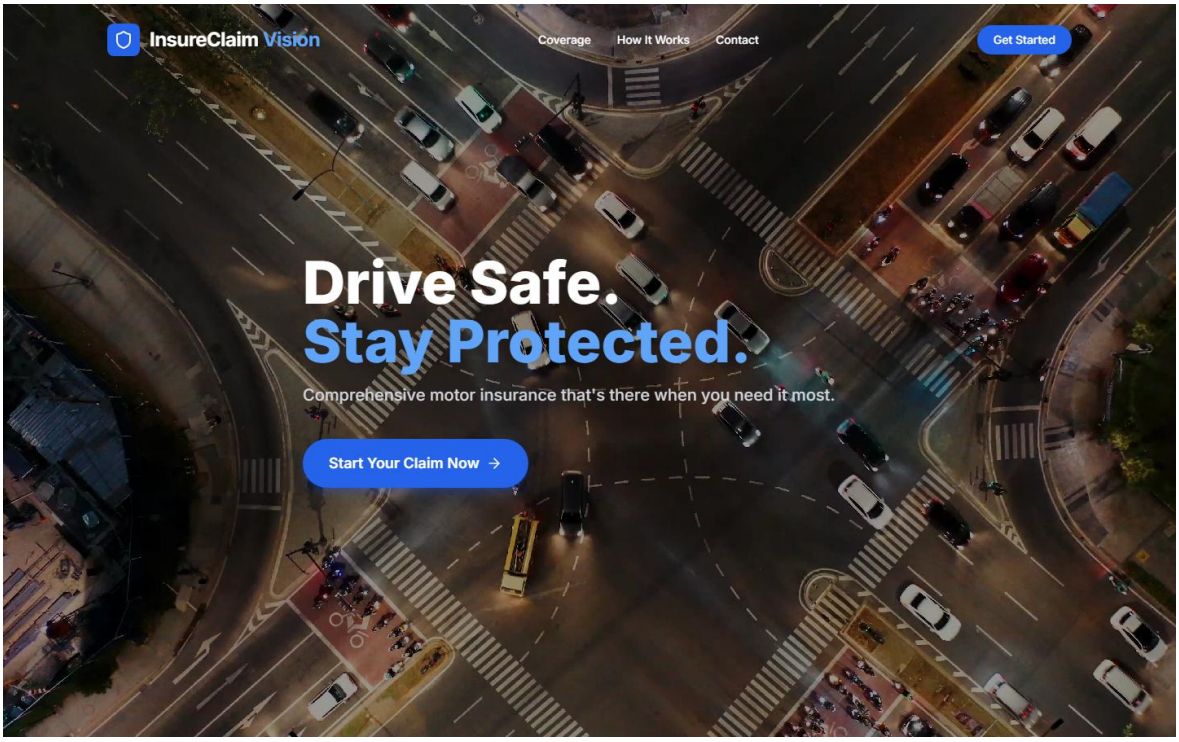
Backend Area	**Main Reviewed Files**
Bootstrap / API	main.py
Schemas	models.py
Perception	cv_processor.py
AI Reasoning	llm_agent.py
Pricing	pricing_service.py, parts_catalog.csv, labor_rates.csv
Storage	database.py (SQLite claims.db)
Experimental	detector.py (YOLOv8 detection engine, not used in the main pipeline)
Configuration	.env.example, requirements.txt, Procfile

6 SCREENSHOTS

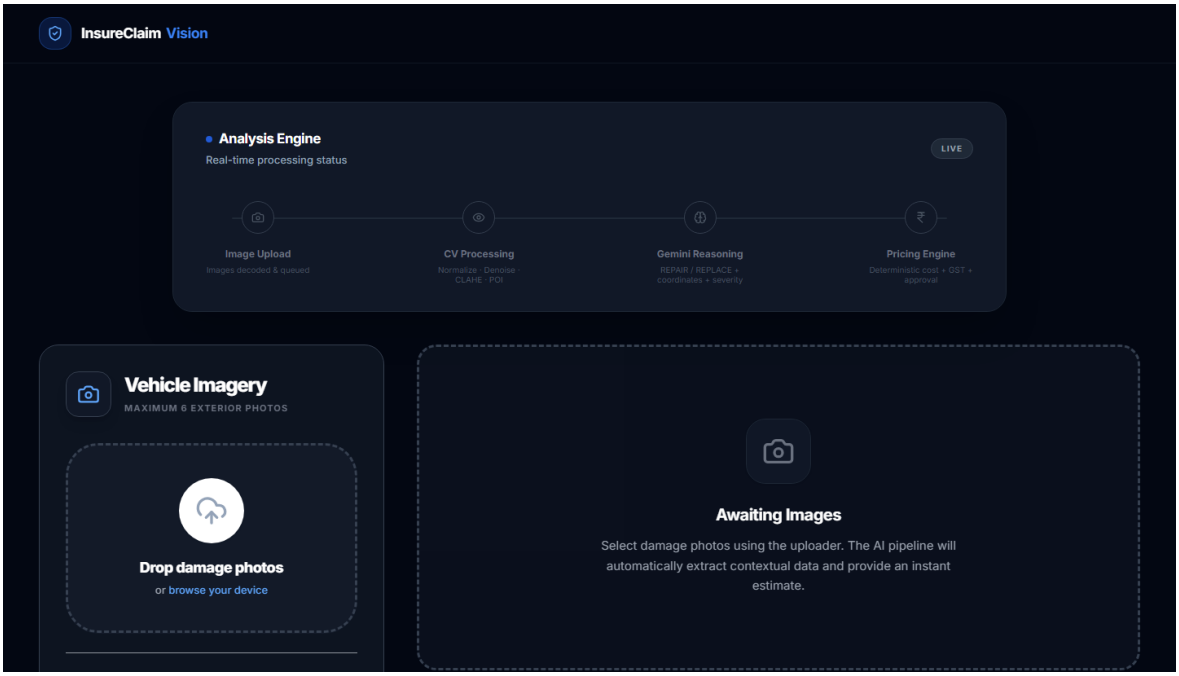
The following screenshots illustrate the key features and user interface of the application. Each screen highlights important functionalities and demonstrates the overall workflow of the system. These visuals provide a better understanding of how users interact with the application in real time.

Below are screenshots of the application showcasing its main features and user interface. These images help in understanding the layout and functionality of the system.

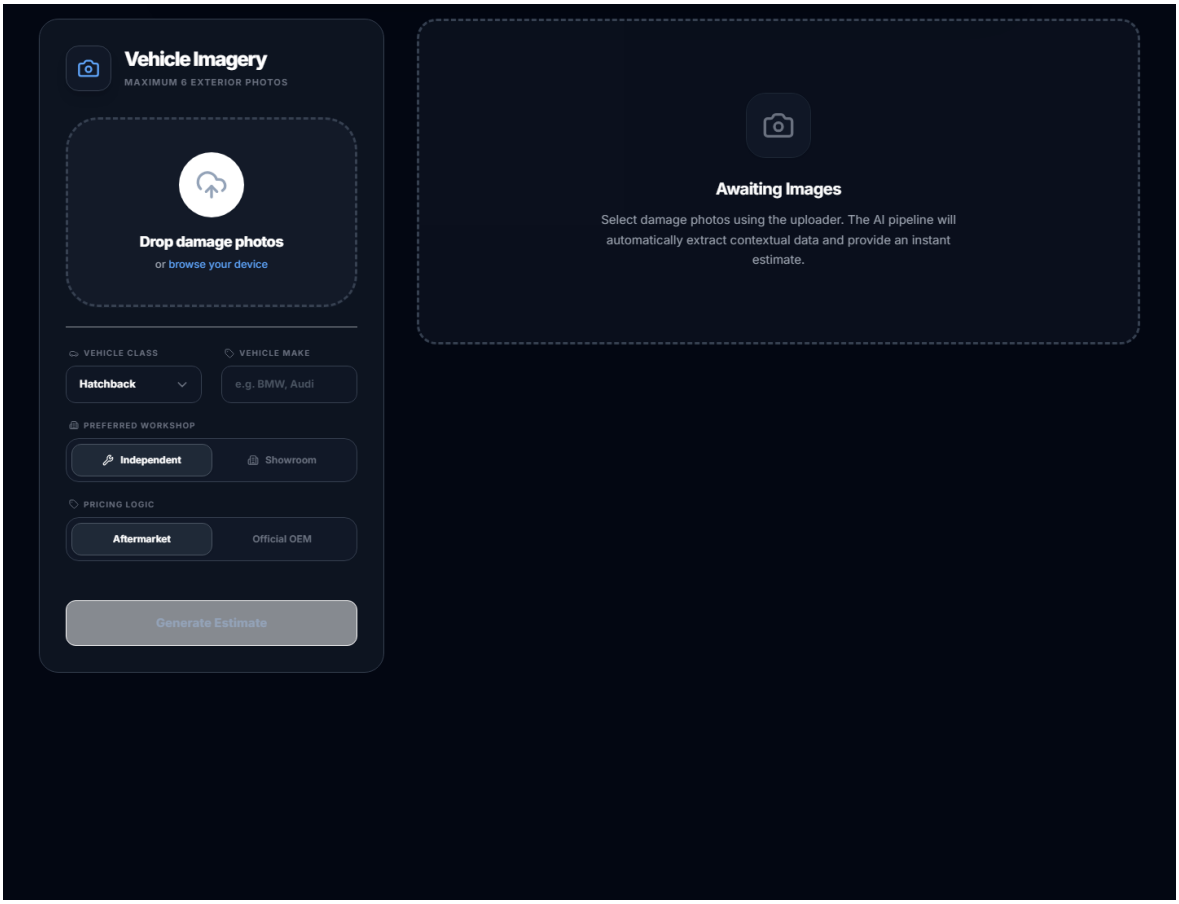
Landing Page Screenshot



Estimator Page Screenshot



Claim Details Form Screenshot



Analysis Result and Approval Screenshot

[Paste screenshot here: result screen with approval badge after analysing a damaged-car photo]

Damage Heatmap and Detection Overlay Screenshot

[Paste screenshot here: heatmap viewer and detection overlay]

Itemised Estimate Screenshot

[Paste screenshot here: estimate panel with parts, labour, GST and grand total]

PDF Claim Invoice Screenshot

[Paste screenshot here: downloaded PDF claim invoice]

7 CONCLUSION

InsureClaim Vision is a strong mini-project because it combines multiple real software engineering concerns in a single cohesive product. It is not limited to static UI work, nor is it only a machine-learning experiment. Instead, it demonstrates end-to-end integration across a modern React interface, an asynchronous Python API, classical computer vision, a multimodal AI model, deterministic business rules, persistence and report generation.

The project's most valuable achievements are the clear separation between AI reasoning and financial calculation, the strict validation of every model output, the visual evidence provided through heatmaps and overlays, and the instant approval decision that can speed up settlement of small claims.

From an academic perspective, the project covers a broad set of modern concepts and shows that the team has worked on interface design, backend design, image processing, prompt engineering, data modelling and system-level integration. The codebase shows deliberate modularisation, typed data contracts and configurable business rules. With further work on accuracy evaluation, multi-image fusion and deployment, the system can be extended into a useful pre-assessment tool for insurers and surveyors.

8 FUTURE ENHANCEMENT

- Fraud detection by cross-referencing damage with historical claim data and duplicate-image checks.
- Pixel-level damage masks using segmentation models such as SAM.
- Integrating the YOLOv8 detection module into the main pipeline for part localisation.
- Fusing results from all uploaded images instead of only the primary image.
- Mobile application for on-site field agents and policyholders.
- Live part prices from workshop and manufacturer APIs instead of static CSV files.
- User accounts, claim history dashboard and surveyor approval workflow.
- Automated testing pipeline and containerised deployment.
- Multi-language interface for regional users.
- Migration to the newer google-genai SDK and model version management.

9 REFERENCES

1. React Documentation: https://react.dev/
2. Vite Documentation: https://vite.dev/
3. TypeScript Documentation: https://www.typescriptlang.org/docs/
4. Tailwind CSS Documentation: https://tailwindcss.com/docs
5. FastAPI Documentation: https://fastapi.tiangolo.com/
6. Pydantic Documentation: https://docs.pydantic.dev/
7. OpenCV Documentation: https://docs.opencv.org/
8. Google Gemini API Documentation: https://ai.google.dev/gemini-api/docs
9. Pandas Documentation: https://pandas.pydata.org/docs/
10. SQLite Documentation: https://www.sqlite.org/docs.html
11. jsPDF Project: https://github.com/parallax/jsPDF
12. Project package dependencies observed in frontend package.json and backend requirements.txt files.